



Universidad Nacional de San Luis
Facultad de Ciencias Físico Matemáticas y Naturales
Departamento de Informática

Práctico Final Integrador
Sistemas Distribuidos y Paralelos

Generación de Cartoons a partir de Imágenes

Juan Cruz Paez
María Lujan Flores Medina

Profesores:
Fabiana Piccoli
Cesar Ochoa

Ingeniería en Informática

2026

Índice general

1. Introducción	1
2. Decisiones de Diseño	3
2.1. Paradigmas y estrategia general	3
2.2. Algoritmo secuencial	3
2.3. Algoritmo implementado en CUDA	3
3. Estadísticas y Métricas de Rendimiento	4
3.1. Métricas de Rendimiento	4
3.2. Configuraciones Experimentales	5
3.3. Entornos de Ejecución	5
3.4. Visualizaciones de Desempeño	6
3.4.1. Speedup Promedio por Configuración	6
3.4.2. Eficiencia Paralela	6
3.4.3. Heatmap de Speedup por Escenario	7
3.4.4. Curva de Escalabilidad	8
4. Conclusiones y Análisis de Resultados	10
4.1. En el entorno local	10
4.2. En el entorno de el cluster	10
A. Resultados del Benchmark en Entorno Local (Compu Juan)	12

Capítulo 1

Introducción

El procesamiento digital de imágenes constituye una de las áreas más demandantes en términos computacionales dentro de la informática moderna. Si bien las bases matemáticas de muchas de sus técnicas datan de siglos atrás, su aplicación práctica a escala masiva solo fue posible con el avance tecnológico del hardware de las últimas décadas, que permitió realizar en tiempo razonable la enorme cantidad de operaciones aritméticas involucradas.

Una imagen digital puede representarse como un arreglo bidimensional de píxeles. En el modelo de color **RGB** (Red, Green, Blue) utilizado en este trabajo, cada píxel contiene tres valores enteros en el rango $[0, 255]$, uno por canal de color. Esta representación permite codificar hasta 16,8 millones de tonalidades distintas, superando la capacidad discriminatoria del ojo humano. Los casos extremos son el negro $(0, 0, 0)$ y el blanco $(255, 255, 255)$.

El objetivo del presente trabajo es la **generación automática de imágenes tipo *cartoon*** a partir de una imagen original en formato RGB. Esta transformación busca simplificar formas, resaltar contornos y reducir variaciones tonales, logrando una apariencia similar a la de una ilustración o dibujo animado. Para ello se aplican dos procesos independientes sobre la imagen original:

- **Posterización:** reduce el rango total de niveles de color a un conjunto más pequeño de valores discretos. Cada píxel pasa a adoptar el valor del nivel más cercano dentro de ese rango reducido. En imágenes a color se posteriza cada canal por separado y luego se suman las imágenes resultantes.
- **Detección de bordes (borroneado):** pipeline de tres etapas secuencialmente dependientes entre sí:
 1. *Filtrado (blur)*: suaviza la imagen para eliminar ruido.
 2. *Resaltado (highlight)*: amplifica las diferencias entre zonas de borde y no-borde mediante aproximación de gradiente (filtros Prewitt, Sobel u otros).
 3. *Umbralado*: binariza el resultado del resaltado para obtener los bordes detectados.

La imagen cartoon final se obtiene sumando la salida de la posterización y la de la detección de bordes.

Dado el volumen de cómputo involucrado, este trabajo explora y compara distintas **estrategias de paralelización** con el fin de reducir el tiempo de procesamiento. Se implementan las versiones del algoritmo:

- **Secuencial:** implementación de referencia, sin paralelismo.
- **Unidad de Procesamiento Gráfico (CUDA):** paralelismo a nivel de threads ejecutando en GPU.

El análisis de desempeño se realiza sobre imágenes de tres tamaños (800×800 , 2000×2000 y 5000×5000 píxeles), con filtros de convolución de 3×3 y 5×5 , y posterizado configurado con 3 y 9 rangos de color. Las métricas utilizadas son el *speedup* y la eficiencia paralela.

Capítulo 2

Decisiones de Diseño

Paradigmas y estrategia general

Para el diseño de los algoritmos paralelos se adoptaron dos paradigmas de forma complementaria:

- **Paradigma de datos** para la distribución de los datos de entrada: dentro de cada tarea, los datos (filas o bloques de la imagen) se reparten entre los workers disponibles.

Dentro de la sección de borroneado, las dos subtarefas (blur y highlight+umbralado) sí presentan dependencia secuencial entre sí, por lo que se implementa un **pipeline de datos**: primero se ejecuta la función de blur sobre toda la imagen y, sobre el resultado, se aplica la función de highlight junto con la operación de umbralado.

Algoritmo secuencial

En todas las versiones del algoritmo se aplicó la técnica de **padding**: se añade un borde auxiliar de píxeles con valor neutro (negro) alrededor de la matriz de entrada. Esto elimina la necesidad de verificar en cada iteración si las celdas vecinas son válidas al calcular el blur y el highlight, simplificando el código y mejorando la localidad de memoria.

Algoritmo implementado en CUDA

Capítulo 3

Estadísticas y Métricas de Rendimiento

Se detallan las métricas y estadísticas de rendimiento recopiladas durante las ejecuciones de prueba del generador de imágenes tipo *cartoon*. Se describen los entornos de prueba, las configuraciones experimentales analizadas y se presentan los gráficos resultantes para visualizar el comportamiento de las distintas implementaciones paralelas.

Métricas de Rendimiento

Para evaluar y comparar cuantitativamente el desempeño de las implementaciones paralelas en relación con la versión secuencial de referencia, se utilizaron las siguientes métricas estándar:

- **Tiempo de ejecución (T_p):** El tiempo total medido en segundos empleado en el procesamiento de la imagen. Para garantizar la precisión de la medición del algoritmo en sí, este tiempo excluye las operaciones de entrada/salida (I/O) en disco (carga de la imagen original y guardado del archivo final procesado).
- **Speedup o Aceleración (S_p):** Representa la ganancia de velocidad obtenida al utilizar recursos paralelos en comparación con el algoritmo secuencial. Se define mediante la fórmula:

$$S_p = \frac{T_1}{T_p}$$

Donde T_1 es el tiempo de ejecución secuencial y T_p es el tiempo de ejecución con la versión paralela utilizando p unidades de recursos (threads o procesos). Un speedup ideal sería uno super lineal.

- **Eficiencia Paralela (E_p):** Mide la fracción de tiempo durante la cual los procesadores asignados son utilizados efectivamente para realizar cálculos útiles. Se define como:

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p}$$

Una eficiencia de 1,0 (100 %) indica que todos los recursos asignados están trabajando a su máxima capacidad sin pérdidas por sobrecarga de sincronización, desbalanceo de carga o comunicación.

Configuraciones Experimentales

Las pruebas de rendimiento se llevaron a cabo variando múltiples parámetros del algoritmo y del entorno para analizar el comportamiento bajo distintas intensidades de cómputo y comunicación:

1. **Resolución de las Imágenes:** Se utilizaron tres resoluciones con distinta cantidad de carga de trabajo:
 - **Baja** (800×800 píxeles): Carga de trabajo ligera (640 mil píxeles).
 - **Media** (2000×2000 píxeles): Carga de trabajo moderada (4 millones de píxeles).
 - **Alta** (5000×5000 píxeles): Carga de trabajo pesada (25 millones de píxeles).
2. **Intensidad del Filtro de Convolución (Borroneado):**
 - **Filtro 3×3 (Radio 1):** Requiere menor cantidad de operaciones por píxel.
 - **Filtro 5×5 (Radio 2):** Requiere mayor cantidad de operaciones de vecindad por píxel.
3. **Rangos de Posterizado:**
 - **3 Rangos (Parámetro 1):** Pocos intervalos de discretización de color.
 - **9 Rangos (Parámetro 2):** Mayor granularidad en el cálculo del color final.
4. **Recursos y Paradigmas Asignados:**
 - **Secuencial:** Ejecución base con un solo thread/proceso (1 recurso).
 - **OpenMP (Memoria Compartida):** Evaluado con 2, 4, 8, 16 y 32 threads en el cluster (y hasta 8 threads en local).
 - **MPI (Memoria Distribuida):** Evaluado con 3, 8, 16 y 32 procesos en el cluster (y hasta 8 procesos en local).
 - **Híbrido (MPI + OpenMP):** Evaluado en combinaciones de procesos $\times$ threads: 3×2 , 3×4 , 3×8 , 4×8 , 8×4 y 16×2 en el cluster (y hasta 5×2 en local).

Entornos de Ejecución

Para contrastar el comportamiento del algoritmo bajo diferentes arquitecturas físicas, las pruebas se corrieron en dos entornos:

- **Entorno Local (Prefijo CJ):** Ejecución sobre una máquina personal multiprocesador.
- **Entorno de cluster (Prefijo RC):** Ejecución sobre el cluster del departamento de informática, que permite evaluar el rendimiento del paso de mensajes entre múltiples nodos de red físicos y la escalabilidad real en sistemas distribuidos.

Visualizaciones de Desempeño

A continuación, se presentan los gráficos generados para representar de manera sintética el comportamiento del sistema.

3.4.1. Speedup Promedio por Configuración

Las siguientes figuras muestran el Speedup promedio (calculado sobre todos los filtros y posterizados) clasificado por el tamaño de la imagen. Las configuraciones están ordenadas numéricamente para apreciar la progresión lógica de los recursos asignados.

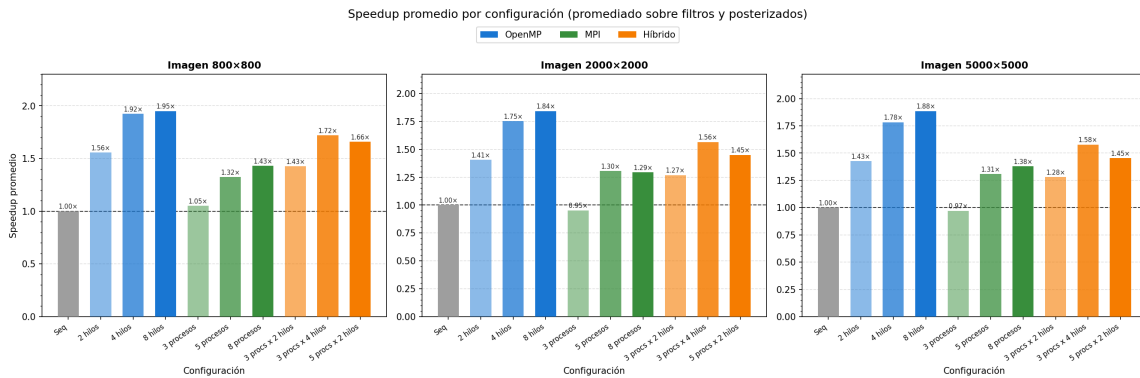


Figura 3.1: Speedup promedio por configuración en Entorno Local para imágenes de 800×800 , 2000×2000 y 5000×5000 .

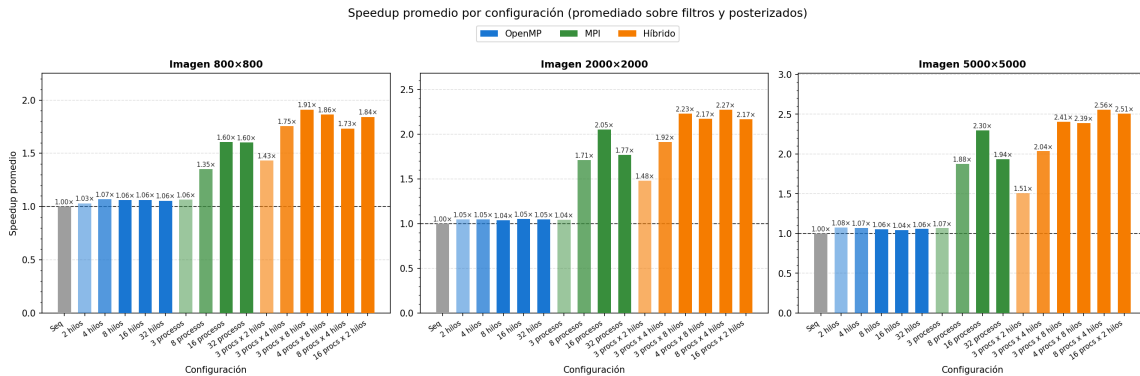


Figura 3.2: Speedup promedio por configuración en Entorno cluster para imágenes de 800×800 , 2000×2000 y 5000×5000 .

3.4.2. Eficiencia Paralela

La eficiencia representa qué tan efectivamente se están usando los recursos agregados. El valor ideal es 1,0.

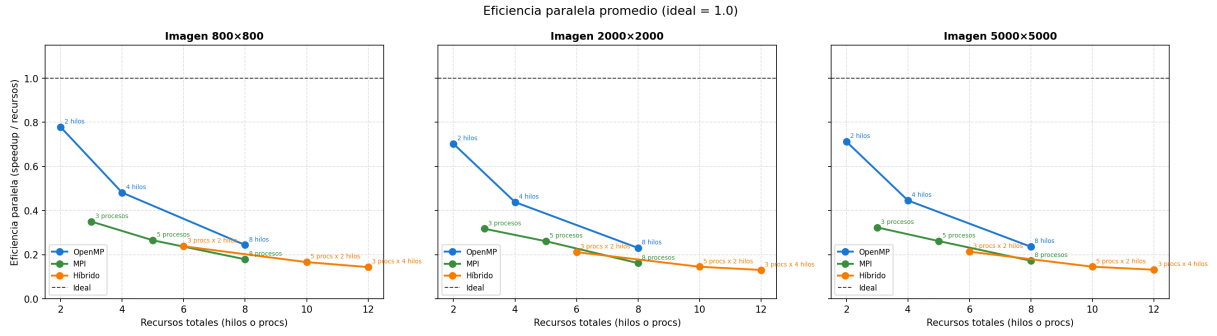


Figura 3.3: Eficiencia paralela promedio en entorno local comparando OpenMP, MPI e Híbrido frente al comportamiento Ideal.

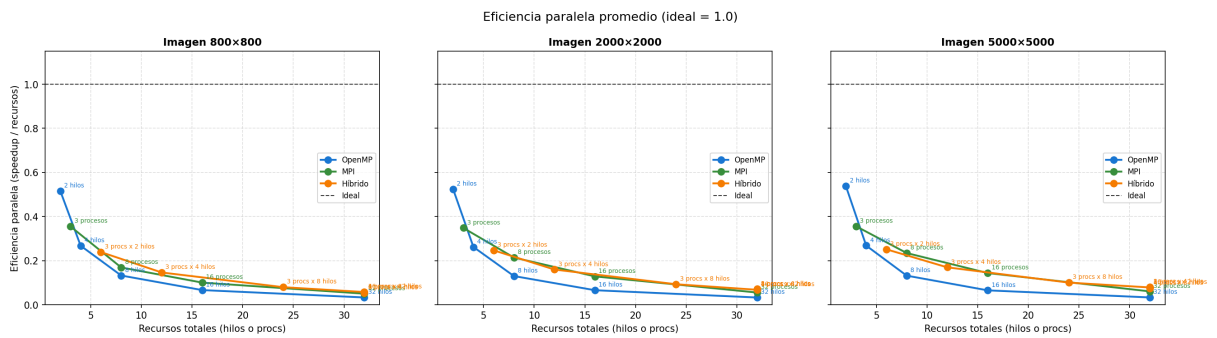


Figura 3.4: Eficiencia paralela promedio en Entorno cluster comparando OpenMP, MPI e Híbrido frente al comportamiento Ideal.

3.4.3. Heatmap de Speedup por Escenario

El heatmap permite cruzar todas las ejecuciones (combinaciones de resolución, filtro y posterización en el eje X) con todas las configuraciones de recursos (eje Y), mapeando el speedup absoluto en una escala de color.

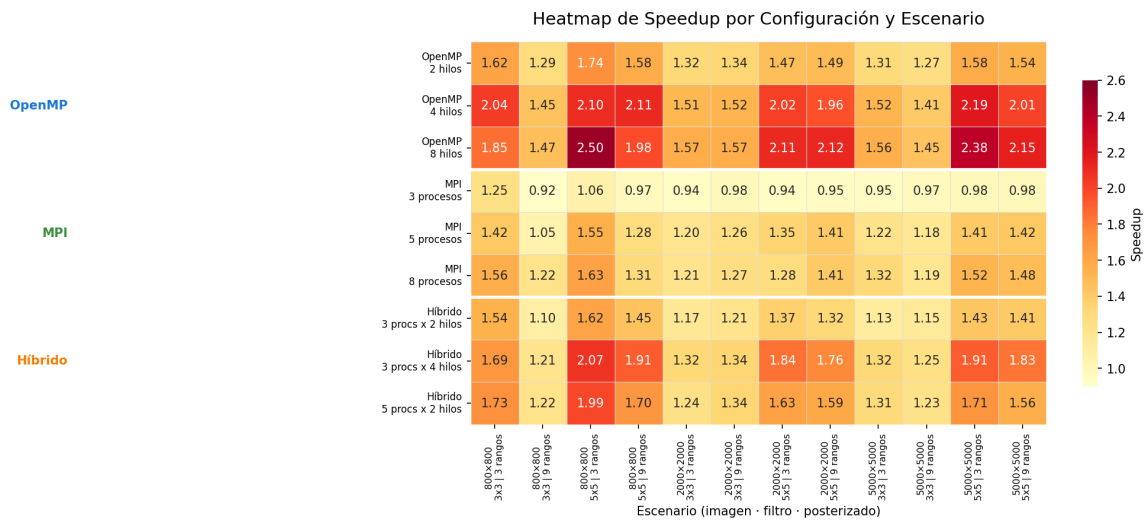


Figura 3.5: Heatmap de Speedup detallado por escenario y configuración en entorno local.

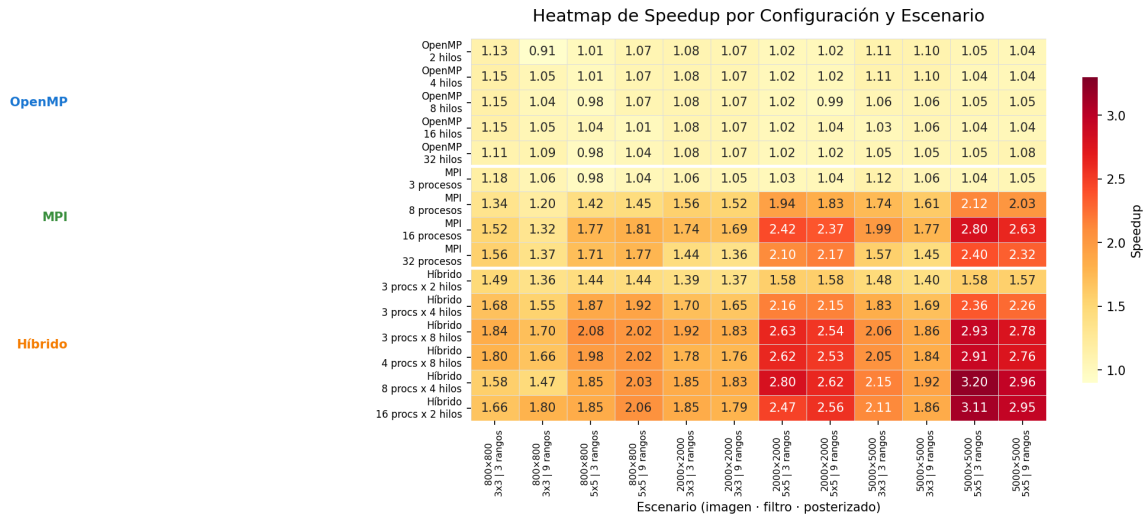


Figura 3.6: Heatmap de Speedup detallado por escenario y configuración en el cluster.

3.4.4. Curva de Escalabilidad

Muestra la evolución del speedup de la mejor configuración de cada paradigma a medida que el problema crece en volumen (de 800×800 a 5000×5000 píxeles).

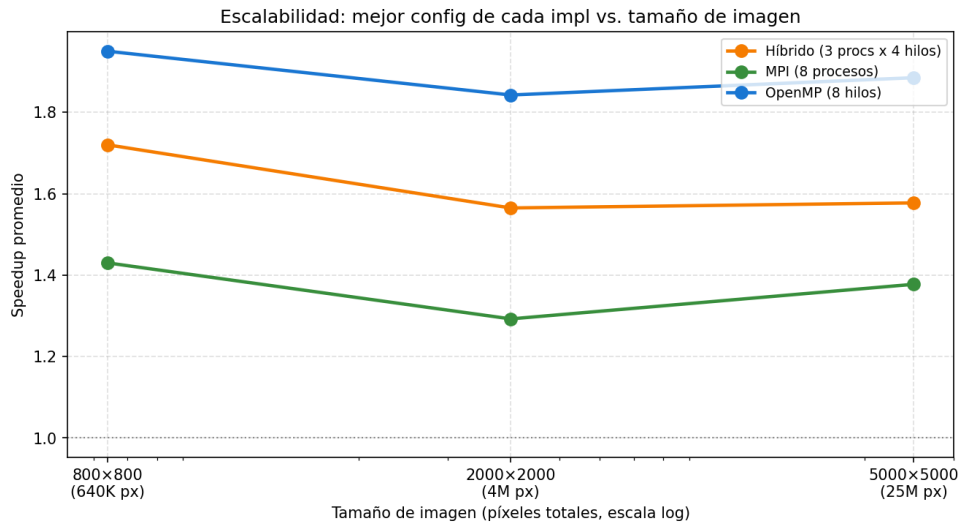


Figura 3.7: Escalabilidad: Speedup promedio de la mejor configuración de cada implementación en función del tamaño de la imagen (local).

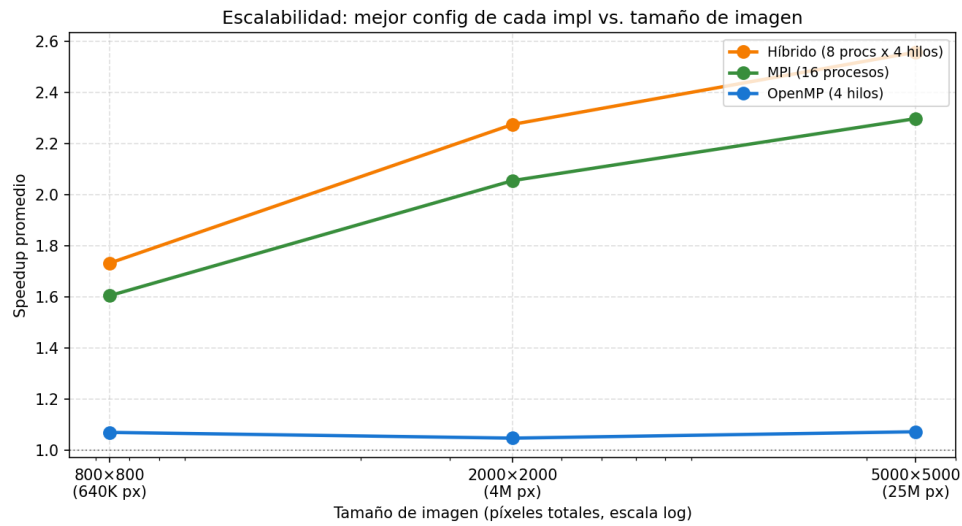


Figura 3.8: Escalabilidad: Speedup promedio de la mejor configuración de cada implementación en función del tamaño de la imagen (cluster).

Nota: Las tablas numéricas de datos crudos obtenidas de cada ejecución se encuentran adjuntas en los Anexos A y ?? al final de este informe.

Capítulo 4

Conclusiones y Análisis de Resultados

Se presenta el análisis de los resultados experimentales obtenidos a partir de las pruebas de rendimiento del generador de *cartoons*. Se contrastan las diferentes implementaciones y se analizan las causas arquitectónicas, de software y de entorno físico que justifican el comportamiento observado en los gráficos.

En el entorno local

El comportamiento de memoria compartida (OpenMP) en el entorno local es el que muestra una gran mejora en los tiempos con respecto a los demás algoritmos. Se asume esto debido a como no se duplican matrices ni se hacen envíos de mensajes, los threads acceden directamente a la imagen cargada en la RAM. De esta forma se obtiene la gran ventaja de OpenMP sobre MPI en todas las configuraciones y resoluciones.

- El techo de rendimiento se encuentra en este caso en los 4 threads, y cuando se hace el filtro 5×5 , creemos que esto es así por que se tiene el triple de multiplicaciones por pixel. Entonces el speedup mejora porque la carga computacional justifica dividir el trabajo en threads

En los algoritmos de MPI y el híbrido consideramos que si bien los resultados mejoran los tiempos del secuencial, estas mejoras son menores que en OpenMP por el tiempo de overhead que se tiene por el pasaje de mensajes. Esto es tiempo en el que OpenMP gana considerable ventaja.

En el entorno de el cluster

El comportamiento de la implementación de memoria compartida (OpenMP) en el cluster muestra que el speedup permanece prácticamente plano (oscilando entre $0,91 \times$ y $1,15 \times$) sin importar que la cantidad de threads aumente de 2 a 32. Se asume que tal comportamiento se atribuye a dos factores: En los entornos de cluster (como SLURM), cuando se ejecuta un proceso multiprocesador de forma nativa sin configurar directivas de asignación específicas (por ejemplo, omitiendo la reserva de múltiples núcleos virtuales por tarea), el sistema operativo limita por afinidad (*affinity binding*) el proceso completo

a un único núcleo físico de CPU. Como consecuencia, aunque la directiva OpenMP en software cree 4, 8, 16 o 32 threads, todos estos threads son forzados por el kernel del sistema operativo a alternarse en el mismo procesador. Esto no solo impide la computación paralela real, sino que añade sobrecarga debido al intercambio de contexto (*context switching*) y a la invalidación de caché de datos.

El diseño del algoritmo divide la carga mediante `#pragma omp parallel sections` asignando un thread al posterizado y otro al borronado. Dado que la convolución bidimensional del borronado (filtro de blur y Sobel) requiere miles de operaciones aritméticas en punto flotante por píxel, esta sección tarda considerablemente más tiempo que el posterizado (que consiste en una división simple por píxel). Al ser la sección de borronado el cuello de botella masivo, y al ejecutarse esta de forma secuencial debido a la restricción de afinidad sobre un solo procesador, el speedup general queda limitado por la Ley de Amdahl a un máximo de $\approx 1,15\times$, haciendo que la adición de threads no tenga impacto real.

A diferencia de OpenMP, las implementaciones que utilizan pasaje de mensajes (MPI) y la version híbrida logran mostrar una *escalabilidad positiva*: Al invocar el programa mediante `mpirun -n p`, se crean p procesos independientes del sistema operativo; así, el planificador del sistema y el entorno MPI distribuyen estos procesos en diferentes núcleos físicos. En cuanto a speedup se observa que tanto para MPI como para la configuración híbrida, el mismo aumenta conforme la imagen es más grande.

Apéndice A

Resultados del Benchmark en Entorno Local (Compu Juan)

En este anexo se presentan las tablas de tiempos, speedup y eficiencia obtenidos en el entorno local.

Imagen: 800x800			
Filtro: 3x3, Posterizado: 3 rangos			
Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.022458	1.00x
GPU (Tile 8)	64 hilos	0.000806	27.86x
GPU (Tile 16)	256 hilos	0.000749	29.98x
GPU (Tile 24)	576 hilos	0.000885	25.36x
GPU (Tile 32)	1024 hilos	0.000778	28.86x

Figura A.1: Resultados del benchmark en entorno local para imagen de 800x800, filtro 3x3 y 3 rangos de posterización.

Filtro: 3x3, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.022597	1.00x
GPU (Tile 8)	64 hilos	0.000803	28.12x
GPU (Tile 16)	256 hilos	0.000772	29.26x
GPU (Tile 24)	576 hilos	0.000902	25.07x
GPU (Tile 32)	1024 hilos	0.000777	29.09x

Figura A.2: Resultados del benchmark en entorno local para imagen de 800x800, filtro 3x3 y 9 rangos de posterización.

Filtro: 5x5, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.054859	1.00x
GPU (Tile 8)	64 hilos	0.002279	24.07x
GPU (Tile 16)	256 hilos	0.001580	34.71x
GPU (Tile 24)	576 hilos	0.001696	32.35x
GPU (Tile 32)	1024 hilos	0.001505	36.46x

Figura A.3: Resultados del benchmark en entorno local para imagen de 800x800, filtro 5x5 y 3 rangos de posterización.

Filtro: 5x5, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.055305	1.00x
GPU (Tile 8)	64 hilos	0.002243	24.66x
GPU (Tile 16)	256 hilos	0.001570	35.23x
GPU (Tile 24)	576 hilos	0.001669	33.13x
GPU (Tile 32)	1024 hilos	0.001500	36.87x

Figura A.4: Resultados del benchmark en entorno local para imagen de 800x800, filtro 5x5 y 9 rangos de posterización.

Imagen: 2000x2000

Filtro: 3x3, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.138506	1.00x
GPU (Tile 8)	64 hilos	0.004862	28.49x
GPU (Tile 16)	256 hilos	0.004366	31.73x
GPU (Tile 24)	576 hilos	0.005789	23.93x
GPU (Tile 32)	1024 hilos	0.003963	34.95x

Figura A.5: Resultados del benchmark en entorno local para imagen de 2000x2000, filtro 3x3 y 3 rangos de posterización.

Filtro: 3x3, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.139167	1.00x
GPU (Tile 8)	64 hilos	0.004920	28.28x
GPU (Tile 16)	256 hilos	0.003849	36.15x
GPU (Tile 24)	576 hilos	0.004844	28.73x
GPU (Tile 32)	1024 hilos	0.003961	35.13x

Figura A.6: Resultados del benchmark en entorno local para imagen de 2000x2000, filtro 3x3 y 9 rangos de posterización.

Filtro: 5x5, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.348351	1.00x
GPU (Tile 8)	64 hilos	0.016063	21.69x
GPU (Tile 16)	256 hilos	0.009996	34.85x
GPU (Tile 24)	576 hilos	0.010129	34.39x
GPU (Tile 32)	1024 hilos	0.009128	38.16x

Figura A.7: Resultados del benchmark en entorno local para imagen de 2000x2000, filtro 5x5 y 3 rangos de posterización.

Filtro: 5x5, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.342613	1.00x
GPU (Tile 8)	64 hilos	0.016041	21.36x
GPU (Tile 16)	256 hilos	0.009830	34.86x
GPU (Tile 24)	576 hilos	0.010162	33.72x
GPU (Tile 32)	1024 hilos	0.008968	38.20x

Figura A.8: Resultados del benchmark en entorno local para imagen de 2000x2000, filtro 5x5 y 9 rangos de posterización.

Imagen: 5000x5000

Filtro: 3x3, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.877222	1.00x
GPU (Tile 8)	64 hilos	0.027111	32.36x
GPU (Tile 16)	256 hilos	0.025208	34.80x
GPU (Tile 24)	576 hilos	0.031109	28.20x
GPU (Tile 32)	1024 hilos	0.024545	35.74x

Figura A.9: Resultados del benchmark en entorno local para imagen de 5000x5000, filtro 3x3 y 3 rangos de posterización.

Filtro: 3x3, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	0.899776	1.00x
GPU (Tile 8)	64 hilos	0.027079	33.23x
GPU (Tile 16)	256 hilos	0.024286	37.05x
GPU (Tile 24)	576 hilos	0.030964	29.06x
GPU (Tile 32)	1024 hilos	0.025088	35.87x

Figura A.10: Resultados del benchmark en entorno local para imagen de 5000x5000, filtro 3x3 y 9 rangos de posterización.

Filtro: 5x5, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	2.154183	1.00x
GPU (Tile 8)	64 hilos	0.091003	23.67x
GPU (Tile 16)	256 hilos	0.057529	37.45x
GPU (Tile 24)	576 hilos	0.066418	32.43x
GPU (Tile 32)	1024 hilos	0.053219	40.48x

Figura A.11: Resultados del benchmark en entorno local para imagen de 5000x5000, filtro 5x5 y 3 rangos de posterización.

Filtro: 5x5, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup
Secuencial	N/A	2.146076	1.00x
GPU (Tile 8)	64 hilos	0.090005	23.84x
GPU (Tile 16)	256 hilos	0.057761	37.15x
GPU (Tile 24)	576 hilos	0.066201	32.42x
GPU (Tile 32)	1024 hilos	0.053592	40.05x

Figura A.12: Resultados del benchmark en entorno local para imagen de 5000x5000, filtro 5x5 y 9 rangos de posterización.